

Intro

안녕하세요, 김종재입니다.

1995. 07. 03

gghs703@naver.com / 01053901348

GitHub Link

<https://github.com/KimJongjae1>



저는 테스트와 로그 데이터를 근거로 문제의 원인을 파악하고, 객관적인 수치를 통해 문제를 해결하는 개발을 추구합니다. 개발 과정에서 Docker를 이용해 로컬 환경에서도 실제 서버와 유사한 환경을 구성하도록 노력하고 있습니다. 재현 가능한 테스트 환경 구축을 중요하게 생각하며, 신뢰성 있는 개발 환경을 통해 완성도 높은 결과물을 만들려고 노력합니다.

프로젝트 요약

- 농산물 경매 사이트 (Freshbid)- WebRTC기반 실시간 농산물 경매사이트/ 수신호,가격예측, 신선도 체크 AI
-역할: 백엔드(상품, 카테고리, 리뷰, 문의 설계 및 구현)/ 수신호 및 가격예측 AI 구현
- 모의투자 사이트 (주식나침반)- 실시간성 주식 모의 투자 시스템/ 800만개의 뉴스에서 각 기업별 핵심 키워드 추출(SPARK)
-역할: 인프라 (Jenkins, Nginx, 웹소켓) / 백엔드 (모의투자 설계 및 구현)

주요 기술



Java

알고리즘(백준 풀 4), 자료구조 기반 구현이 탄탄하고, 서비스 로직을 안정적으로 작성 가능한 수준



SpringBoot

REST API 설계, Spring Security + JWT 인증, JPA 기반 CRUD/검색, 트랜잭션 처리



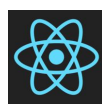
MySQL

ERD/스키마 설계, 인덱스 고려 쿼리 작성, 실행계획 기반 성능 점검 경험



Vue

AI 보조로 CRUD 화면과 API 연동을 구현한 경험, 컴포넌트 설계/상태관리는 보완 중



React

AI 보조로 기본 화면 구성과 API 연동 경험, 프로젝트 구조화와 상태관리 경험은 추가 확보 중

교육

삼성 청년 소프트웨어 아카데미(SSAFY)

2025. 01. 01 - 2025.12.18

경력

셀트리온제약 (생산관리)

2022. 06. 27 - 2024. 09. 26 (2년 3개월)

학력

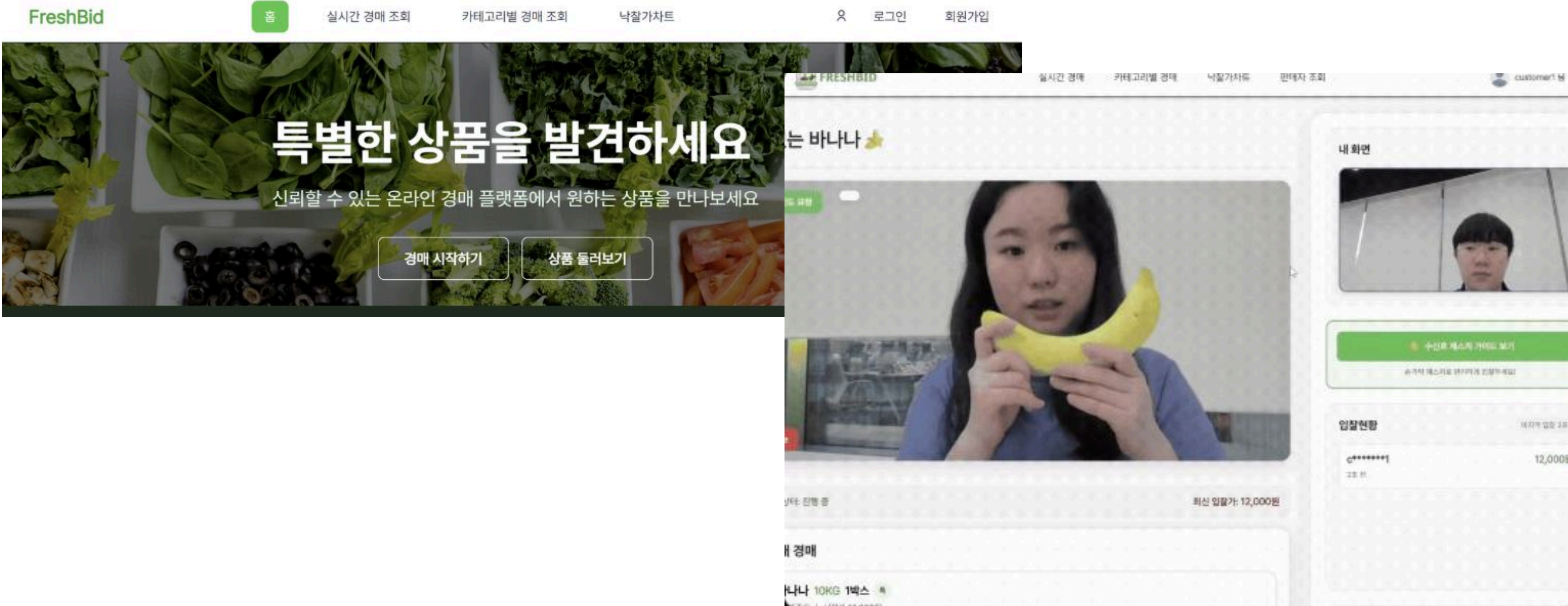
건국대학교 서울캠퍼스

2015- 2021 (융합생명공학과)

자격증

정보처리기사

취득일:2025.12



서비스

- 농산물 실시간 온라인 경매 사이트
- WebRTC 기반 라이브 경매
- 수신호 인식 /신선도 체크/ 가격 예상 AI

백엔드 기술 스택



인프라



역할

BackEnd

- 상품/카테고리 도메인 설계 및 API 구현
- 후기/리뷰 API 구현
- 낙찰가 차트 데이터 API 설계 및 가격 예측 도입
- WebRTC 라이브 중 입찰 수신호 자동 인식 도입
- 맡은 부분의 DB 설계/ 구축

FrontEnd

- 차트 페이지 구현
- 후기/리뷰 페이지 구현

작업 기간

2025.07.07 ~ 2025.08.18 (6주)

구성원-6명

- FE 2명
- BE 김종재 외 3명

프로젝트 회고

- 프로젝트를 통해 **Git, jira**를 이용하여 **에자일 방식의 팀 프로젝트 수행**
- Docker/ jenkins 사용으로 자동 배포
- 수신호 AI를 실시간 경매에 적용하면서 성능 최적화 경험
- Swagger 사용으로 프론트 개발자와 쉽게 소통하고 API 테스트 수행
- JPA를 사용함으로써 간단한 CRUD는 Repository 메서드 호출로 간단 해짐. 만약에 복잡해지면 QueryDSL 사용.
- 테스트의 중요성을 실감했음. 특히 CI/CD에 큰 관심을 갖게됨.

프로젝트 중 직접 구현한 서비스

1. 수신호 인식 및 자동 입찰



- YOLO와 Mediapipe를 이용하여 실제 경매에서 사용한 수신호를 인식하여 자동으로 호가를 올려줌.

동작원리

- MediaPipe Hands
실시간 영상에서 손을 검출하고 21개 랜드마크로 손 위치, 자세를 추적해 ROI(손 영역)를 안정적으로 추출.
- Resize, 전처리
추출한 손 ROI를 모델 입력 규격에 맞게 리사이즈해 추론 품질을 일정하게 유지.
- YOLOv8 추론, 결과 출력
YOLOv8로 수신호 클래스를 분류하고, 바운딩 박스 좌표, 라벨, 신뢰도(conf)를 반환해 화면에 시각화.

2. 낙찰가 차트 및 가격 예측



- Prophet 모델(사계열 예측)을 이용한 낙찰가 차트 및 가격 예측 구현
- 3개년의 가락시장 데이터를 크롤링 하여 기본 데이터를 구성하고 이후에는 자동으로 데이터를 수집하여 매일 새벽 2시마다 새로운 가격 예측을 해줌.
- Redis 캐싱 도입으로 낙찰가 차트 응답 속도 3.0초 → 0.2초로 실시간 데이터 캐싱 적용, 차트 로딩 시간 90%이상 단축

3. 카테고리/상품/리뷰 API 설계·구현

- 공통: Spring Boot + Spring Security 인증, 표준 CommonResponse 래핑, Swagger/OAS 문서화, Soft delete 플래그로 안전 삭제, DTO 분리로 검증/응답 필드 관리.

카테고리 (/categories)

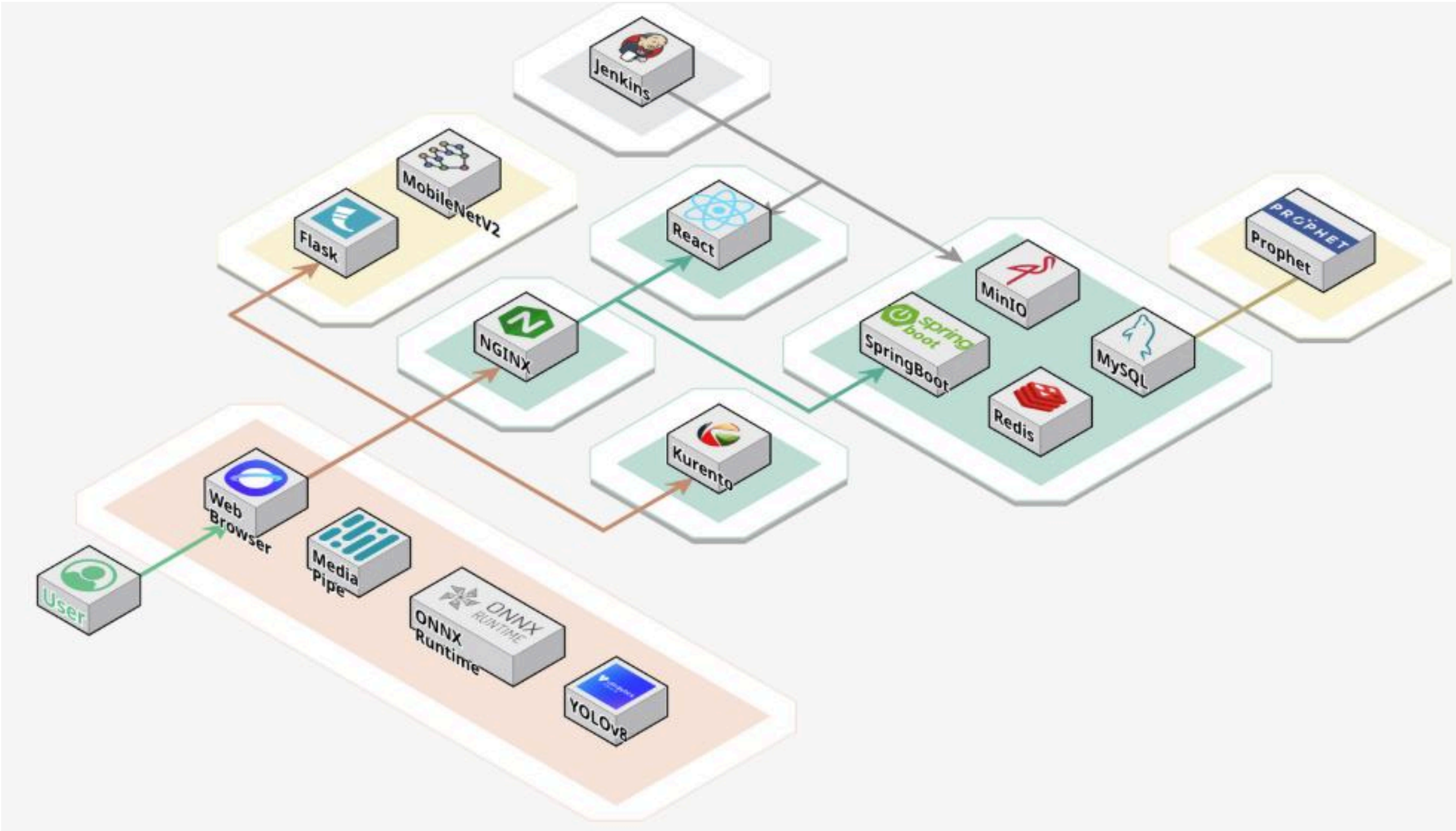
- “상위-하위” 2단 구조. 상위 카테고리 지정 → 저장” 흐름. 상위 ID가 틀리면 등록을 아예 막아서 엉뚱한 부모를 갖는 잘못된 트리가 생기지 않게 함.
- 이름/상위ID로 조회 가능. “캐시” 버튼으로 상·하위/등급 정보를 Redis에 담아 빠르게 꺼냄.

상품 (/auction/product)

- “판매자 계정만 등록/수정/삭제 가능”으로 잠금. 본인 상품이 아니면 거절.
- 사진은 MinIO에 올려서 링크만 저장. 이름/카테고리/등급/무게 범위로 필터 검색, 정렬+페이징네이션 지원.

판매자 리뷰 (/auction/review)

- “배송 완료된 주문” + “주문자나 판매자 본인”만 리뷰 작성 가능. 같은 주문에 중복 리뷰 금지.
- 대댓글은 한 단계만 허용(자기 리뷰엔 못 답글). 작성자만 수정/삭제 가능, 삭제는 숨김 처리.
- 평점/작성일로 정렬해 페이지 단위로 내려줌.



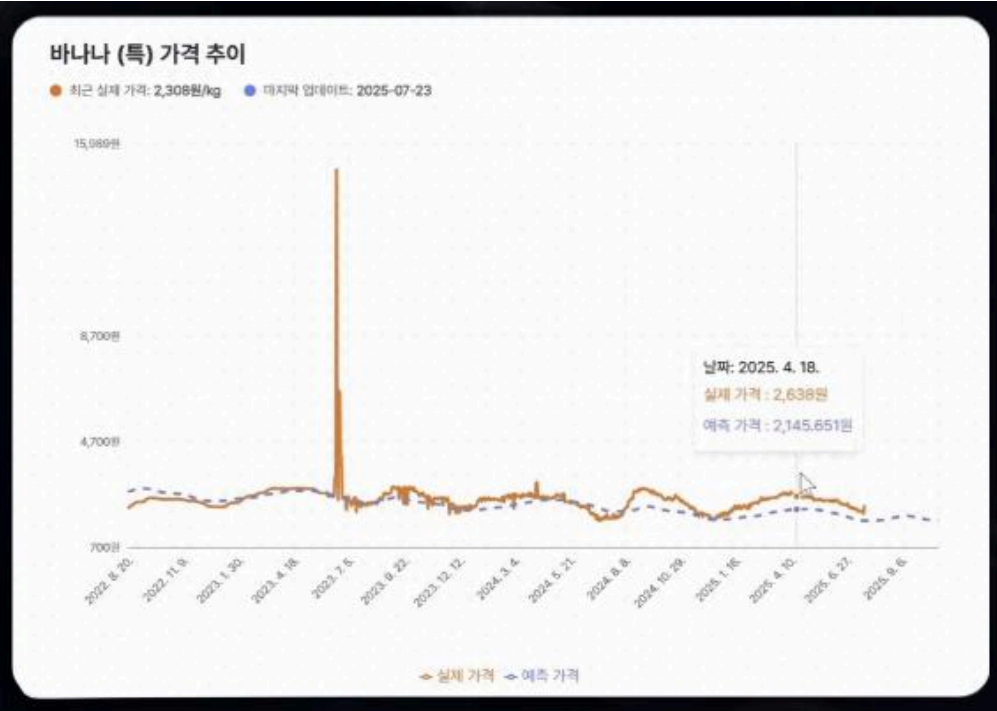
트러블 슈팅

- 1. 수신호 AI 성능 최적화 (3FPS → 17FPS)
 - 검증: Ultralytics YOLOv8 val로 테스트셋 4,854장 평가 → F1=0.91, 추론 지연 수 ms → 모델 이상 없음.
 - 원인: 병목은 브라우저 파이프라인(캡처-인코딩-전송-디코드-전/후처리-추론-렌더).
 - 개선: 입력 160×160 경량화 + WASM 적용 + 캡처 루프 수정 → F1=0.90 유지, WebRTC 1-2 → 11-13 FPS(기록).
- 2. 차트 API 속도 개선 (3초 → 0.2초)
 - Redis 캐싱 도입: 카테고리/등급 및 차트 데이터 캐싱
 - 배치 처리: 일일 배치로 예측 데이터 미리 생성

```
TS typescript
// front/src/hooks/useGestureModel.ts
if (isInferringRef.current) {
  return []; // 이미 추론 중이면 스킵
}
isInferringRef.current = true;
```

추론 중복 방지 코드와 수신호 인식 예시

```
J java
// back/src/main/java/FreshBid/back/service/impl/CategoryCacheServiceImpl.java
@Override
public void cacheSuperCategories(List<PriceDataRequestDto> superCategories) {
  String json = objectMapper.writeValueAsString(superCategories);
  redisTemplate.opsForValue().set(SUPER_CATEGORIES_KEY, json, CACHE_TTL);
}
```



배치 처리 코드와 실제 낙찰가 차트 예시

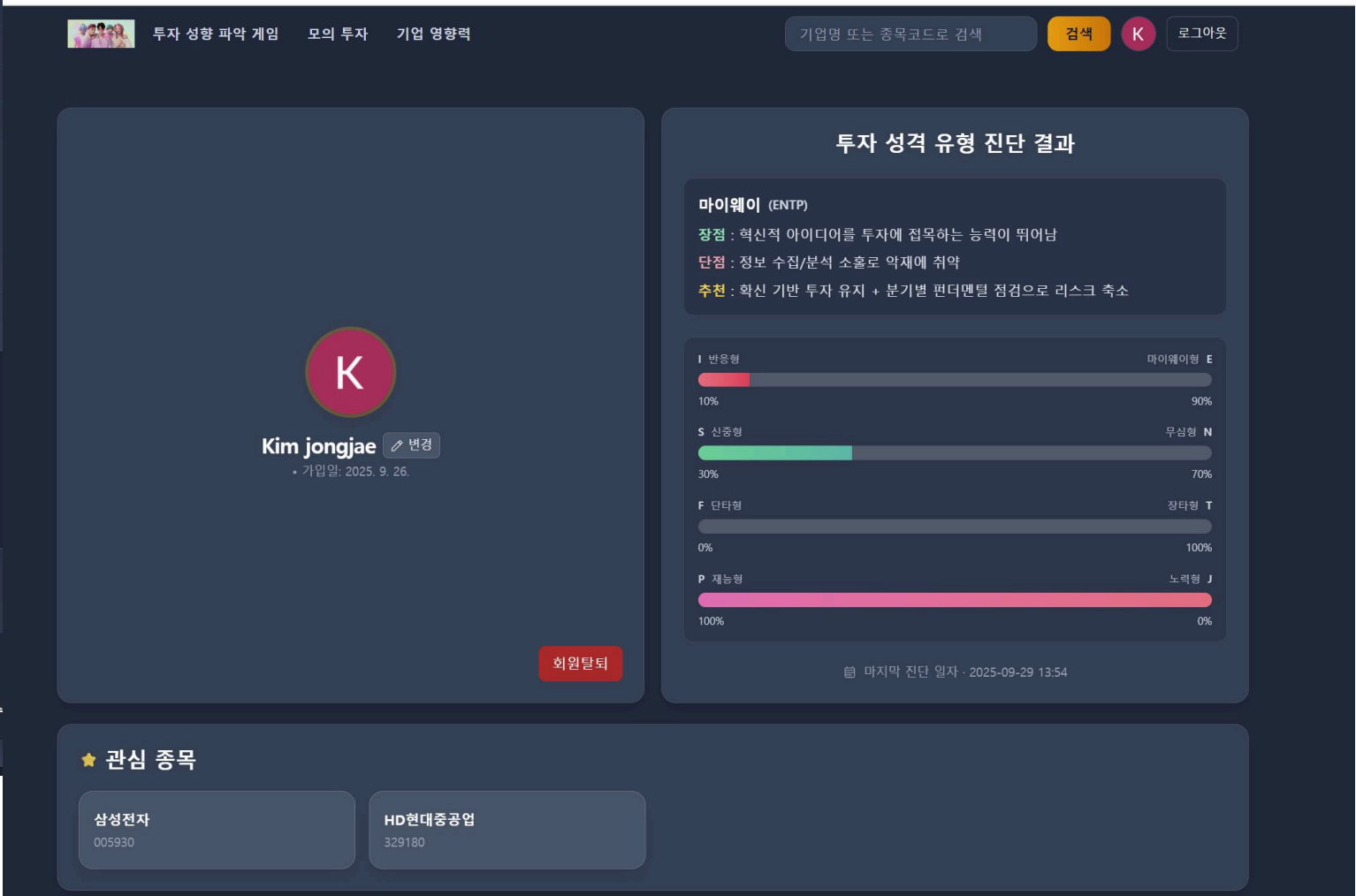
- 3. JWT 인증/인가 통합
 - 문제: 프론트/백엔드/웹소켓 간 토큰 전달 방식 불일치로 인증 실패 및 연결 오류 발생
 - 백엔드 토큰 처리 표준화 (만료 시 RefreshToken 자동 갱신)
 - 프론트 Axios 인터셉터로 모든 요청에 토큰 자동 포함
 - WebSocket 연결 시 동일한 검증 로직 적용

```
J java
// back/src/main/java/FreshBid/back/filter/JwtAuthenticationFilter.java
if (authorizationHeader != null && authorizationHeader.startsWith("Bearer ")) {
  String token = authorizationHeader.substring(7);
  jwtTokenProvider.validateToken(token);
}
```

백엔드 필터에서 토큰 표준화 및 자동 갱신

```
TS typescript
// front/src/api/axiosInstance.ts
axiosInstance.interceptors.request.use((config) => {
  const token = useUserStore.getState().accessToken;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

프론트 Axios 인터셉터로 모든 요청에 토큰 자동 포함



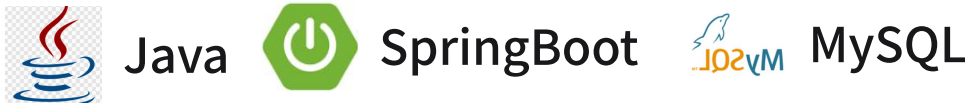
서비스

실시간성 주식 모의 투자 시스템

투자성향 분석 게임

800만건의 뉴스에서 각 기업별 핵심 키워드 추출

백엔드 기술 스택



인프라



역할

Infra

자동빌드/배포 파이프라인 구성 (Webhook → Jenkins → Docker Compose)

Nginx 리버스 프록시 설정 (HTTPS 리다이렉트, /api/*, /oauth2/* 라우팅)

Certbot을 이용한 SSL 인증서 적용 및 자동갱신 확인

EC2 서버 관리 (네트워크, Docker 서비스, 볼륨/로그 관리)

서버 내 웹소켓 설정(한국투자증권에서 실시간 주식 정부 수신)

BackEnd

모의 투자 설계

1. TradeController-매수/매도, 자산조회
2. StockController-실시간 데이터 및 히스토리 조회
3. AccountService/StockService-자산 및 손익계산/주 가처리 및 데이터 수집

관련 DB 설계

작업 기간

2025. 08. 25 - 2025. 09. 29 (5주)

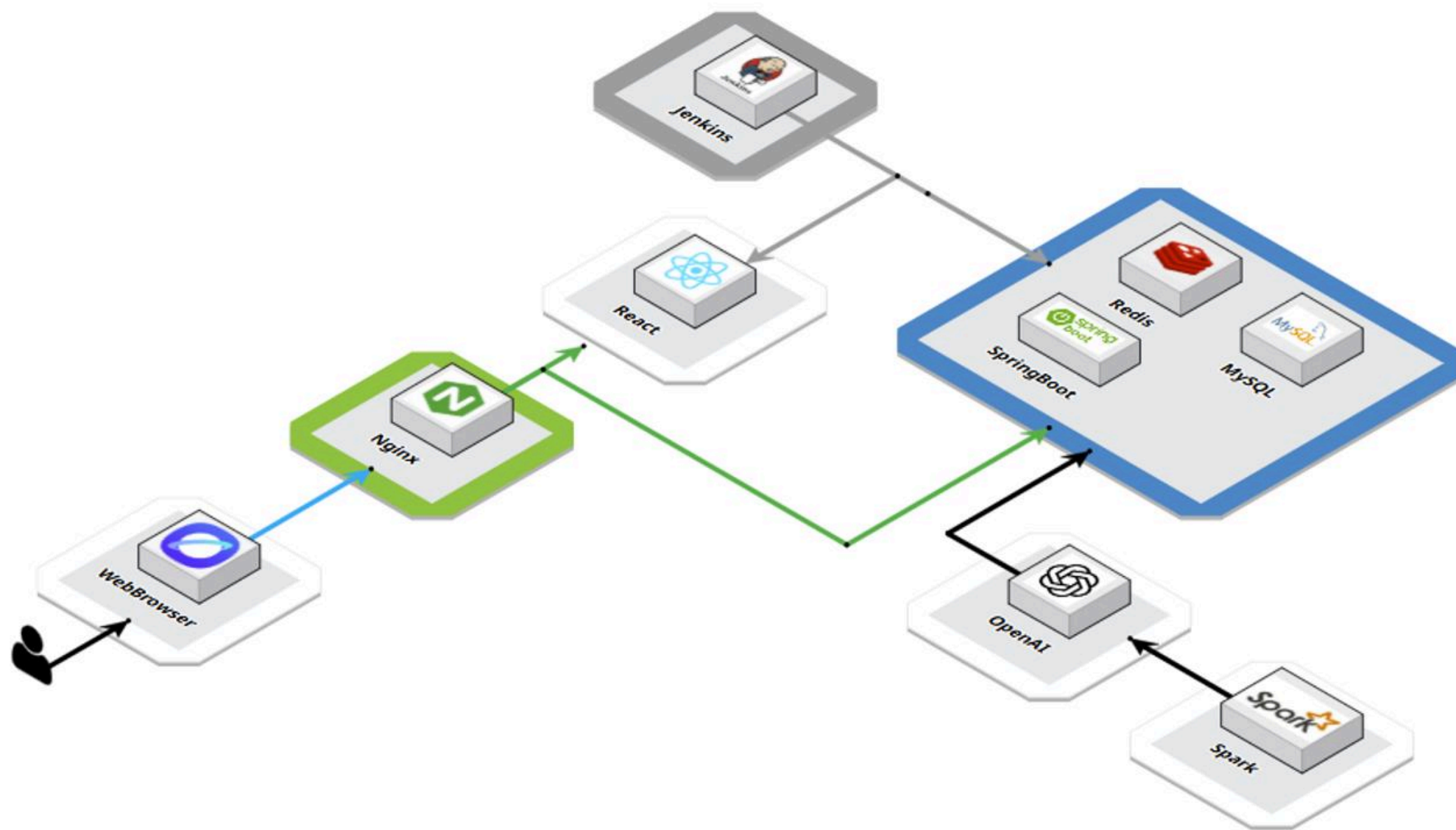
구성원-6명

FE 2명, BE 3명

Infra&BE (김종재)

프로젝트 회고

- 인프라에 대한 지식
- 리눅스 명령어, 젠킨스, Nginx에 대한 이해
- 모의투자시에 설계를 잘 못해서 시간을 많이 낭비함. 백엔드 설계의 중요성을 배움.
- 테스트에 대한 중요성- 단위, 통합 테스트의 중요성과 단순 API테스트로는 부족하다는 것을 배움.
- 앞으로 설계와 테스트를 위주로 개발을 하고싶다는 생각을 했음.



트러블 슈팅-인프라

1. 하드코딩

- Front/Back 모두 8080/8765 하드코딩 (로컬호스트 개발 기준)
- 팀장으로써 이런 부분을 초기에 확인 못 한점. 또한, 코드를 쓸 때, 항상 서버 가동을 염두에 두어야함을 배움.

2. Https에 대한 이해도 부족

- OAuth(구글/카카오)를 하려면 Https를 사용해야하지만 http로 초기 서버를 설정해버림.
- Cerbot과 Let's encrypt를 배움. HTTPS는 내가 서버가 맞는지 확인하는 서버키와 각 API를 암호화하는 세션키로 있음. 이후, 설정 완료함.

테스트

단위 테스트

- AccountServiceImplTest.java: DB연결 없이 보유수량·평단 계산, 거래 없을 때 0/0 반환 (리포지토리 Mock).
- AccountControllerTest.java: MockMvc standalone, /api/trade/holding/{ticker} 정상 요청 시 200·DTO 매핑 확인(시큐리티 필터 배제).
- 통합 테스트 (@Tag("integration"))
 - AccountTest.java: AccountService 실제 빈/DB 경로 확인.
 - StockInfosTest.java: StockInfosRepository/Service 쿼리·트랜잭션 동작.
 - StockItemsTest.java: StockItemRepository 기본 조회/단건 조회.
 - StockRealtimeTest.java: 실시간 종목 페이지네이션/조회.
 - TradeTest.java: 매수/매도 처리, 자산 조회, 거래내역 조회.